Slotbook

A booking engine that never double-books | Take-home assignment report

1. Links and summary

Item	Details
GitHub repository	https://github.com/pedapallisuresh/slotbook *(confirm this is the final public URL)*
Live URL	Not deployed (optional extension). Runs locally with one command: `docker compose up --build`
Author / brief	Suresh Pedapalli. Brief shared by @Rachit, dated 4 Nov 2026; submitted within the 72-hour window.
Stack	TypeScript end to end. Hono API on Node 20, PostgreSQL 16 (no Redis), Luxon for time zones, plain HTML/JS front end, Vitest, Playwright, Docker Compose.

What was built

- **Host dashboard**: seeded login, home IANA time zone, event types (duration 15-120, capacity, buffers 0-30, minimum notice, horizon), weekly hours with several windows per day, date overrides, upcoming bookings with cancel-with-reason, notification feed, embed snippet.
- **Guest flow**: host page, event page with a month calendar and time zone switcher, 5-minute hold, confirm, private manage link to view, reschedule or cancel.
- **Booking engine**: held, confirmed, cancelled and expired states; capacity counts holds and confirmed bookings only; idempotent holds; notifications on confirm, reschedule and cancel.
- **Extras**: daily cap and per-guest limit, rate limiting, JSON logs with request IDs, /health and /metrics, embeddable widget, load-test script, Playwright test and GitHub Actions workflow.

Headline results

Check	Result
Integration tests (real Postgres 16)	14 of 14 pass in about 1 second, including a 100-way race on one slot
Browser end-to-end test (Chromium)	1 of 1 passes: book, open manage link, reschedule, cancel
Load test (20 virtual users, 10 s)	241 requests/s, p95 145 ms, 0 unexpected errors; 200 holds created, 2,211 correctly rejected; never more than 5 active holds on a capacity-5 slot

Load numbers come from one machine running Node and Postgres together, so treat them as indicative, not a capacity claim.

How to run

```
docker compose up --build   then open http://localhost:3000  (host login: demo@slotbook.dev / demo1234)
```

Tests: `npm test` (integration), `npm run e2e` (browser), `npm run loadtest`. Full instructions are in README.md. Brief assumptions are listed at the end of section 4.

2. Architecture

One Node process serves the pages and the JSON API. All business rules live in a small booking engine that talks to PostgreSQL; Postgres is the only source of truth, so correctness never depends on memory, caches or timers in the app.

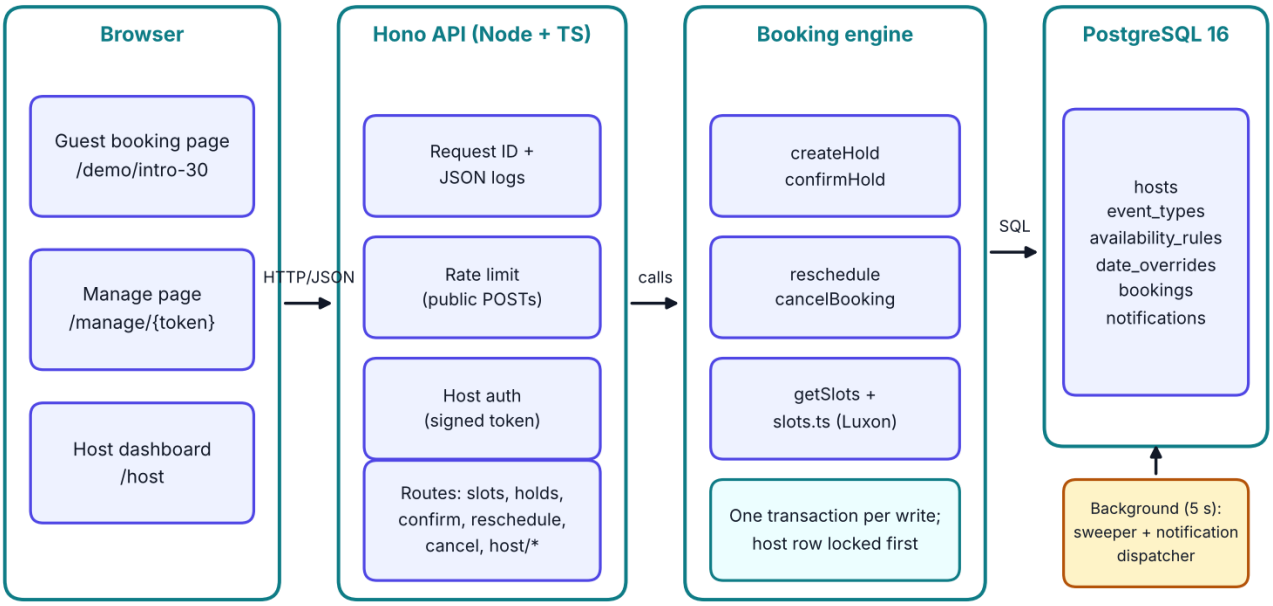
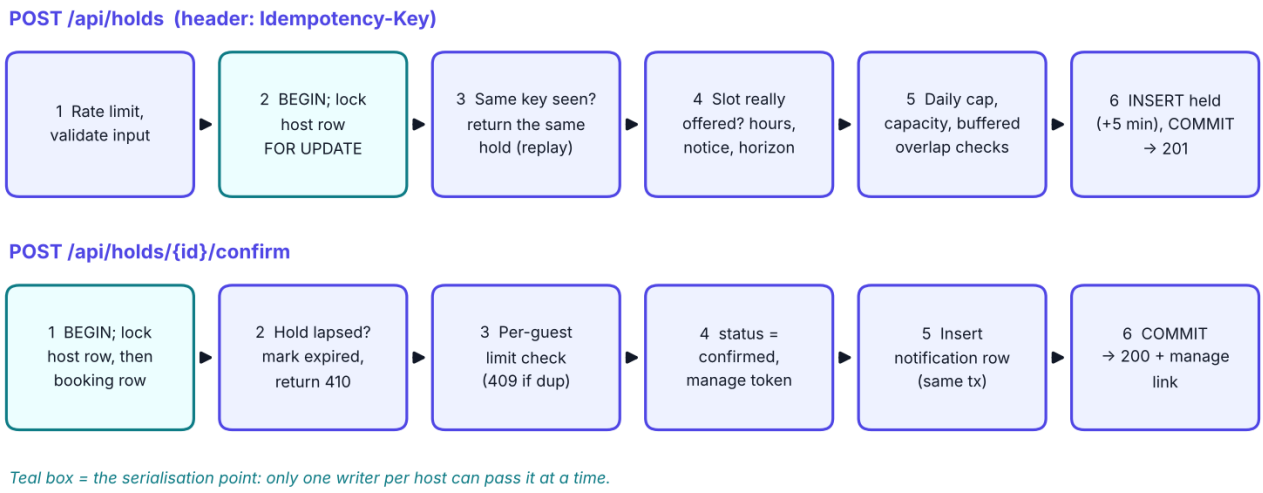


Figure 1. Components. A background loop every 5 seconds marks lapsed holds as expired and stamps notifications as delivered.

How a booking request flows



Teal box = the serialisation point: only one writer per host can pass it at a time.

Figure 2. The two write paths. Every write takes the host row lock first, so writers for one host queue up inside Postgres while different hosts never wait for each other.

- **Read path** (GET slots): load the host's hours and overrides, generate candidate slots in the host's zone, then remove slots that are full or overlap an active booking. Only reads, no locks.
- **Reschedule**: same lock-then-check path as a new hold, but excluding the booking being moved, in one transaction. If the new slot is taken, nothing changes.
- **Cancel**: lock the booking row, set cancelled with the reason, write a notification, commit. Repeating it is harmless.

3. Data model

Six tables. Constraints that protect the rules are in the database itself, not only in application code.

Table	Key columns	Constraints
hosts	id, slug, name, email, tz (IANA), password_hash, daily_cap	slug and email unique; password stored as salted scrypt hash; daily_cap CHECK > 0 (null = unlimited)
event_types	id, host_id, slug, name, duration_min, capacity, buffer_before_min, buffer_after_min, min_notice_min, horizon_days	UNIQUE(host_id, slug); CHECK duration 15-120, capacity >= 1, each buffer 0-30
availability_rules	host_id, weekday (ISO 1-7), start_time, end_time	CHECK weekday 1-7 and end_time > start_time; several rows per day allowed (e.g. lunch break)
date_overrides	host_id, on_date, is_off, start_time, end_time	A row means day off (is_off) or custom hours for that date, replacing weekly rules
bookings	id, event_type_id, host_id, start_at, end_at, **blocked_start, blocked_end**, status, hold_expires_at, guest_name, guest_email, manage_token, cancel_reason, idempotency_key	status CHECK in (held, confirmed, cancelled, expired); manage_token UNIQUE; idempotency_key UNIQUE; index on (host_id, blocked_start, blocked_end)
notifications	id, booking_id, kind (confirmed / rescheduled / cancelled), payload (jsonb), created_at, delivered_at	Append-only sink, written in the same transaction as the booking change

Definitions used everywhere

- **Active booking** = status is confirmed, or status is held and hold_expires_at is later than the database clock. Capacity and overlap checks count only active bookings, so expired and cancelled ones never block anyone.
- **blocked_start / blocked_end** = the slot widened by its before and after buffers, stored when the booking is made. Two bookings clash when these ranges overlap, unless they are the same event type at the same start (a group session sharing seats).
- Times are stored as timestamptz (UTC instants). Local wall-clock times exist only in availability_rules and date_overrides, interpreted in the host's zone.

4. Key decisions

1. Serialise writers per host with a row lock (SELECT ... FOR UPDATE on the host)

Alternatives: advisory locks; SERIALIZABLE transactions with retry; a Postgres exclusion constraint on time ranges; Redis lock (not allowed).
Why: one lock covers capacity counting for group sessions *and* overlap across different event types including buffers. An exclusion constraint cannot express capacity above 1, and SERIALIZABLE needs retry loops. Easy to reason about and to test.
Trade-off: one writer at a time per host. Fine for this scale; other hosts are unaffected.

2. Expiry is decided at read time; a sweeper only tidies up

Alternatives: cron-only expiry; database trigger.
Why: a hold older than its deadline is ignored by every query, so correctness never depends on a job running on time. The 5-second sweeper then sets status to expired for tidy lifecycle and metrics.
Trade-off: expired rows linger up to 5 seconds as 'held' in the table.

3. Store the buffered window on each booking

Alternative: compute buffers from the event type at query time.
Why: if a host later edits buffers, old bookings keep the protection they were made with, and overlap becomes one range comparison.
Trade-off: two extra columns.

4. Idempotency key is checked after taking the host lock, backed by a UNIQUE index

Alternative: check the key first, then lock.
Why: checking first lets two simultaneous first requests both miss, then race. Locking first makes a retry see the original hold and return it unchanged.
Trade-off: replays also queue behind the lock (cheap).

5. Store UTC; generate slots from local wall-clock hours using IANA zones (Luxon)

Alternatives: store fixed UTC offsets; recurrence-rule libraries; store local times only.
Why: a fixed offset is wrong across daylight saving changes. Local hours in the host's zone stay correct and are converted per date. Display uses the browser's Intl API.
Trade-off: nonexistent local times in the spring-forward gap rely on Luxon's normalisation.

6. Notifications as a table written in the same transaction (outbox style)

Alternatives: send after commit; external queue service.
Why: the event exists if and only if the booking change committed. A dispatcher stamps delivered_at, so lag is measurable at /metrics.
Trade-off: the sink is a table; real email or webhook delivery is not built.

7. Hono API with plain HTML/JS pages instead of Next.js or a React app

Alternatives: Next.js, React single-page app.
Why: one process, fast start, trivial Docker setup, and the brief values a clean reliable core over visual polish. The pages are still responsive and keyboard operable.
Trade-off: less componentised front end; React is a next step.

8. Private manage link as an unguessable token; host login as a seeded password

Alternatives: guest accounts; expiring signed links; magic-link email for hosts.
Why: guests need no account. The token is 24 random bytes (base64url) with a UNIQUE index. Host passwords are salted scrypt hashes with HMAC-signed 24-hour tokens.
Trade-off: anyone holding a manage link can manage that booking, and links do not expire.

Assumptions made

- Slot step equals the event duration (a 30-minute event offers slots every 30 minutes within each window).
- In a group session each guest is one booking row; capacity counts rows. The daily cap counts booking rows per host-local day.
- Hold length is 5 minutes (fixed constant); one seeded demo host with two event types (a 1:1 and a group of 5).
- Per-guest limit is checked at confirmation, case-insensitively by email, for the same event type among confirmed future bookings.

5. Reliability: failure cases, handling and proof

Each row names the failure, how the system handles it, and the automated test that demonstrates it (file test/concurrency.test.ts unless noted).

Failure case	How it is handled	Proof
Two guests grab the same slot at once	Host row lock, then capacity check inside one transaction; the loser gets 409.	100 parallel holds on a 1:1 slot: exactly 1 wins. Load test: max 5 active on a capacity-5 slot.
Group session over-filled	Holds plus confirmed bookings at the same start are counted under the lock.	40 parallel holds, capacity 5: exactly 5 succeed.
Network retry, double click, lost response	Idempotency-Key header (unique); a replay returns the original hold, no second row.	Same key twice returns the same hold id.
Guest abandons the form	Hold is ignored after 5 minutes; sweeper marks it expired.	Expired hold frees the slot; sweeper test sets status to expired.
Guest confirms after the hold lapsed	Confirm re-checks the database clock, marks expired, returns 410.	Test: confirm of a lapsed hold gives 410 and status expired (this test found and fixed a rollback bug).
Clocks misbehave (browser or server)	All expiry and active-booking decisions use Postgres now(); the browser clock only drives the countdown display.	Tests move hold_expires_at in the database rather than waiting.
Reschedule target already taken	Move happens in one transaction; any failure rolls back and the old slot stays.	Collision returns 409, old start unchanged; a free slot then succeeds.
Back-to-back bookings and buffers across event types	Buffered ranges compared with a range-overlap test across all event types of the host.	Different event type at same time: 409; inside the buffer: 409; after the buffer: allowed.
Cancel then rebook; expired holds	Cancelled and expired bookings are excluded from every count.	Cancel then rebook the same slot succeeds.
Daily cap and duplicate guest	Checked under the host lock (cap at hold and reschedule, duplicate at confirm).	Daily-cap test (third booking gets 409); per-guest test (second active booking gets 409).
Daylight saving changes	Slots built from local wall-clock hours per date.	test/slots.test.ts: New York 09:00 maps to 14:00Z, then 13:00Z after the change.
Forged or stale slot sent to the API	Server re-derives offered slots (hours, overrides, notice, horizon) and rejects others with 422.	Notice and horizon test; day-off and custom-hours test; HTTP smoke test.
Bots flooding public endpoints	Per-IP rate limit on public POSTs (default 120 per minute); holds expire in 5 minutes.	Not covered by an automated test (see gaps).
Database error mid-request	Every write is one transaction; errors roll back everything including notifications.	By design; no partial writes seen in the load test (0 unexpected errors).

6. Time zones

Generating slots. For each date in the booking range the engine takes the host's weekly windows (or that date's override), builds the window's start and end as local wall-clock times in the host's IANA zone, and walks forward in steps of the event duration. Slots earlier than now plus the minimum notice, or beyond the horizon, are dropped; slots that are full or overlap an active booking are removed. Each slot is stored and transferred as a UTC instant.

Displaying slots. The guest page formats every instant with the browser's Intl API in the selected zone (default: the browser's zone), so changing the switcher re-renders the same instants. The calendar groups slots by the guest's local date, which can differ from the host's date.

Worked example across a daylight-saving change

Host zone America/New_York, hours Monday to Friday 09:00. Daylight saving began on Sunday 8 March 2026, when clocks moved from 02:00 to 03:00 (EST, UTC-5, became EDT, UTC-4).

Host local date	Local slot	Offset	UTC instant stored	Guest in Asia/Kolkata (UTC+5:30)
Fri 6 Mar 2026	09:00	EST, UTC-5	14:00Z	19:30
Mon 9 Mar 2026	09:00	EDT, UTC-4	13:00Z	18:30

The host's slot stays at 09:00 on both days, but the stored UTC instant moves by an hour, and the Kolkata guest sees 19:30 then 18:30. A system that stored 'every weekday 14:00Z' would show the host a 10:00 meeting after the change. The automated test reproduces exactly these two instants.

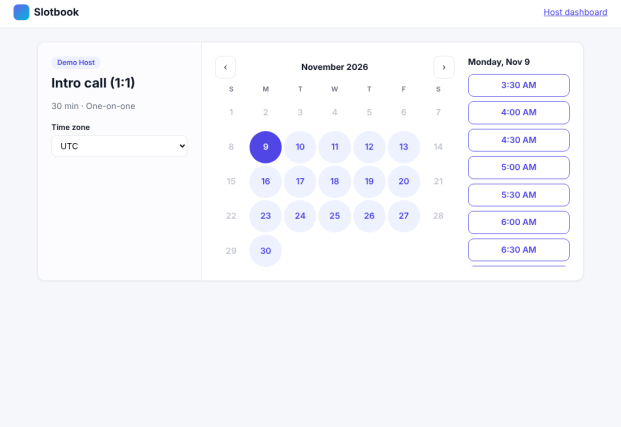
Edge cases

- Time zone names are validated against the IANA database when a host saves them.
- Hours that fall inside the spring-forward gap (for example 02:30 on 8 March in New York) do not exist; Luxon normalises them and the result is not separately tested.
- A guest in a far-away zone may see a slot on a different calendar date than the host; the picker groups by the guest's own date.

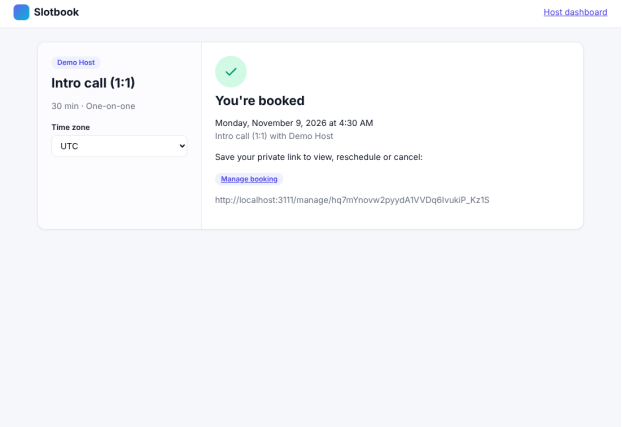
7. Optional extensions

Extension	Status	How it fits the architecture
Daily cap and per-guest limit	Built	Extra checks inside the same locked transaction (cap at hold and reschedule, duplicate guest at confirm). Daily cap is a host setting.
Abuse protection	Partly	Per-IP rate limit middleware on public POSTs; short holds limit slot hogging. No CAPTCHA; limiter is in memory.
Observability	Built	JSON log line per request with a request id; GET /health checks the database; GET /metrics reports bookings by status, hold expiries swept, notification lag and pending count.
Load test	Built	scripts/loadtest.mjs fires concurrent holds at 40 group slots and reports throughput, p50/p95/p99 and error rate (section 8).
End-to-end tests and CI	Built	Playwright test of the guest flow; .github/workflows/ci.yml runs typecheck, integration tests and the e2e test against a Postgres service. The workflow has not yet been run on GitHub.
Embeddable widget	Built	The booking page supports ?embed=1 (no site chrome); the dashboard has Copy embed for an iframe snippet.
Accessibility	Partly	Labelled controls, real buttons, keyboard-operable calendar with descriptive aria-labels, live status regions. No screen-reader audit yet.
Waitlist, reminders, calendar invites (.ics), host webhooks, round-robin, external busy times, live deployment	Not built	Listed as next steps in section 10. The transactional notification table is the natural base for reminders and signed webhooks.

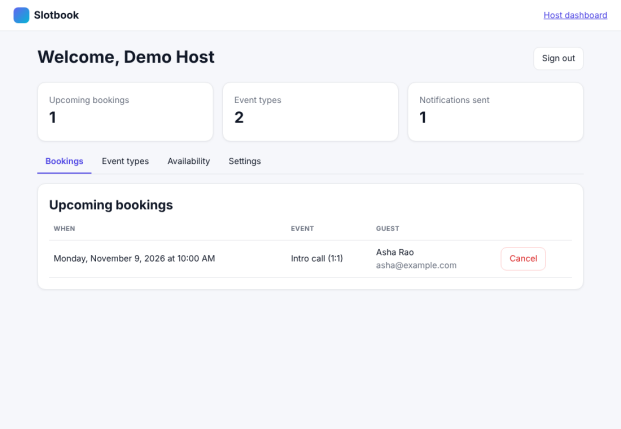
Product tour (screenshots)



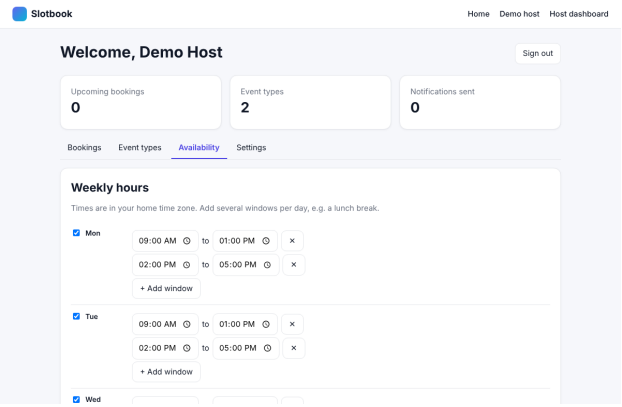
Guest page: calendar, time list, time zone switcher.



Confirmation with private manage link.



Host dashboard: stats, tabs, upcoming bookings.



Weekly hours editor with several windows per day and date overrides.

8. Testing

Run: `npm test` runs 14 integration tests against a real Postgres (no mocks). `npm run e2e` runs the browser test against a running app. `npm run loadtest` runs the load test.

Test	What it proves
100 parallel holds on one 1:1 slot	Exactly one succeeds; no double booking under a race
40 parallel holds on a capacity-5 session	Exactly five succeed
Same Idempotency-Key twice	Same hold returned; no duplicate row
Expired hold and cancelled booking	Both free the slot for the next guest
Reschedule collision, then free slot	409 and old slot kept; then the move succeeds
Notification rows	Confirm and reschedule each wrote an event
Per-guest limit	Second active booking by the same email is rejected
Host daily cap	Booking beyond the cap on a local day is rejected
Sweeper	Lapsed holds become 'expired'
Buffers across event types	Overlap and buffer clashes rejected; clear slot accepted
Confirm after expiry	410 and status expired
Slots: daylight saving (slots.test.ts)	New York 09:00 is 14:00Z then 13:00Z
Slots: overrides (slots.test.ts)	Day off removes slots; custom hours replace weekly hours
Slots: notice and horizon (slots.test.ts)	Too-soon and too-far slots are not offered
Playwright: guest flow (e2e/guest.spec.ts)	Book, open manage link, reschedule, cancel in a real browser

Results

Integration: 14 of 14 passed on PostgreSQL 16. Browser end-to-end: 1 of 1 passed in Chromium. The app was also started under Docker Compose on the author's Windows machine and served the pages.

Load test (scripts/loadtest.mjs)

Setting / metric	Value
Workload	20 virtual users for 10 s, each sending POST /api/holds on random slots among 40 group slots (capacity 5), unique Idempotency-Key per request
Environment	Single Linux sandbox machine, Node and Postgres 16 on the same host, one Node process, default pool
Requests / throughput	2,411 requests, 241.1 requests per second
Latency	p50 75.3 ms, p95 145.2 ms, p99 195.3 ms
Outcomes	201 created: 200 (= 40 slots x 5 seats); 409 rejected: 2,211; unexpected errors (5xx, network): 0 (error rate 0%)
Invariant check	After the run, the maximum active bookings on any slot was 5 (the capacity)

Latency is high-contention by design: nearly every request fights for the same few hosts' lock. The point of the run is the invariant (never above capacity) and zero unexpected errors, not peak throughput.

9. Using AI

I used Claude (Anthropic) throughout: reading and breaking down the brief, choosing the locking design, writing the schema, slot generator, booking engine, API, pages, tests, Docker files, load-test script and this report. I ran the project on my own machine and fed the errors back.

Key prompts

- Explain the assignment and plan the core (correctness before features).
- Write the schema and booking engine so that concurrent bookings can never exceed capacity, with a concurrency test against real Postgres.
- Add the HTTP API, host dashboard, guest and manage pages, then audit everything against the brief and fix gaps.
- Fix errors from my own runs (Docker connection problems, an upgrade error, a port clash).

Where the AI was wrong or misleading, and how it was caught

- First file-creation script wrote nothing because of a shell quirk; noticed from the empty output and re-run.
- An early group-capacity test had the wrong expected value; caught when reviewing the test and rewritten.
- New database columns were added to the migration, but Docker only runs migrations on a fresh volume, so an existing database crashed the notification loop (visible in my log). Fixed with a self-healing schema step at startup, tested against an old-schema database.
- A new catch-all route /:host swallowed /metrics; found by calling every URL after the change and fixed by reordering.
- Writing the confirm-after-expiry test exposed that the 'mark expired' update was rolled back by the error; fixed so the status change is kept while still returning 410.
- The AI's first answers were not verified until run; load numbers and test counts in this report come from actual runs, not from the AI's estimates.

What I changed or reviewed by hand

[Add 2 to 3 sentences in your own words here: which parts you read line by line (for example the locking and the daylight-saving code), what you changed, and what you chose not to use.]

Raw prompts and a short note are in AI_LOG.md in the repository.

10. Known gaps and next steps

Known gaps

- **Not built:** waitlist, reminders, calendar invites, host webhooks, round-robin, external busy times, live deployment.
- **Rate limiter** is in memory and keys on the X-Forwarded-For header or a single shared bucket when no proxy is present; it has no automated test and no CAPTCHA.
- **Notifications** are rows in a table; the dispatcher only stamps delivered_at. No email or webhook is sent.
- **Manage links** never expire; anyone with the link can cancel or move that booking.
- **Hosts** cannot yet edit or delete event types from the UI. Reschedule keeps the same event type.
- **Front end** is plain HTML/JS, not React; accessibility is keyboard-tested only, with no screen-reader audit.
- **Time:** nonexistent local times in the spring-forward gap rely on Luxon and are not separately tested. CI workflow not yet run on GitHub. Load numbers are from a single machine.
- **Throughput:** the per-host lock serialises writes for one host. That is the right trade for correctness, but a very busy single host would be the first bottleneck.

With another day

- Waitlist with time-limited offers, then reminders at 24 h and 1 h and .ics attachments, both built on the notification table.
- HMAC-signed host webhooks with retries and a delivery log.
- Deploy (Fly.io or Render plus Supabase) with a seeded demo host.
- Proper per-client rate limiting, a test for it, and a screen-reader audit.
- A database-level exclusion constraint for capacity-1 event types as a second safety net behind the lock.
- Edit and delete for event types; expiring manage links; React front end.